

```
#####
#
#   COURS : CHIMIE & BIOLOGIE
#   Théorie · Démonstrations · Exemples · Code Python natif
#
#####
#
# COMMENT LIRE CE COURS ?
#
# Chaque notion suit ce plan :
# [THÉORIE]      - L'idée, pourquoi ça existe
# [DÉFINITION]   - La règle formelle
# [DÉMONSTRATION] - Preuve ou raisonnement pas à pas
# [EXEMPLE]      - Exemple concret
# [CODE PYTHON]  - Programme qui illustre la notion
# [PIÈGE]        - Erreur classique à éviter
#
#####
```

PARTIE 1 – CHIMIE GÉNÉRALE

1. STRUCTURE DE L'ATOME ET TABLEAU PÉRIODIQUE

[THÉORIE]

Tout ce qui nous entoure est composé d'atomes. Un atome est la plus petite unité d'un élément chimique qui conserve ses propriétés. Comprendre la structure de l'atome, c'est comprendre pourquoi les éléments réagissent entre eux, pourquoi le sodium explose dans l'eau, ou pourquoi l'or est si stable.

Un atome est constitué de trois types de particules :

- Les PROTONS : chargés positivement (+), situés dans le noyau
- Les NEUTRONS : neutres, situés dans le noyau
- Les ÉLECTRONS : chargés négativement (-), orbitant autour du noyau

[DÉFINITION]

NUMÉRO ATOMIQUE (Z) : nombre de protons dans le noyau.
→ Définit l'élément chimique (Z=1 → Hydrogène, Z=6 → Carbone...)

NOMBRE DE MASSE (A) : nombre de protons + neutrons.
 $A = Z + N$ (N = nombre de neutrons)

NOTATION STANDARD :

${}^A_Z X$ ex : ${}^{12}_6 C$ (carbone-12 : 6 protons, 6 neutrons)

ISOTOPES : atomes d'un même élément (même Z) mais avec un nombre de neutrons différent.
ex : 1H (protium), 2H (deutérium), 3H (tritium) – tous des hydrogènes

CONFIGURATION ÉLECTRONIQUE :

Les électrons occupent des couches (niveaux d'énergie) notées K, L, M...
Règle de remplissage (Aufbau) : on remplit par ordre d'énergie croissante
 $1s \rightarrow 2s \rightarrow 2p \rightarrow 3s \rightarrow 3p \rightarrow 4s \rightarrow 3d \rightarrow 4p \dots$

Capacités maximales : $s \rightarrow 2e^-$, $p \rightarrow 6e^-$, $d \rightarrow 10e^-$, $f \rightarrow 14e^-$

RÈGLE DE L'OCTET :

Les atomes tendent à avoir 8 électrons sur leur couche de valence (comme les gaz nobles), ce qui explique la formation des liaisons.

[EXEMPLE]

Carbone (C) : Z=6, A=12, N=6
Configuration : $1s^2 2s^2 2p^2 \rightarrow$ 6 électrons au total
Couche de valence : $2s^2 2p^2 \rightarrow$ 4 électrons de valence
Le carbone peut former 4 liaisons (pour atteindre l'octet).

Sodium (Na) : Z=11, A=23
Configuration : $1s^2 2s^2 2p^6 3s^1$
1 seul électron de valence $\rightarrow$ il le cède facilement $\rightarrow Na^+$ très stable.

[CODE PYTHON]

Calculateur de configuration électronique simplifiée

```
def config_electronique(Z):
    """Retourne la configuration électronique d'un atome de numéro atomique Z."""
    sous_couches = [
        ('1s', 2), ('2s', 2), ('2p', 6), ('3s', 2), ('3p', 6),
        ('4s', 2), ('3d', 10), ('4p', 6), ('5s', 2), ('4d', 10),
        ('5p', 6), ('6s', 2), ('4f', 14), ('5d', 10), ('6p', 6)
    ]
    config = []
    reste = Z
    for nom, capacite in sous_couches:
        if reste <= 0:
            break
        electrons = min(reste, capacite)
        config.append(f"{nom}{'012345678910'[electrons]} if electrons < 11 else str(electrons)")
        config.append(f"{nom}^{electrons}")
        reste -= electrons
    # Version lisible
    config_lisible = []
    reste = Z
    for nom, capacite in sous_couches:
        if reste <= 0:
            break
        electrons = min(reste, capacite)
        config_lisible.append(f"{nom}^{electrons}")
        reste -= electrons
    return " ".join(config_lisible)

elements = {1: "H", 2: "He", 6: "C", 8: "O", 11: "Na", 17: "Cl", 26: "Fe"}

print(f"{Z:>4} | {'Élément':>8} | Configuration électronique")
print("-" * 55)
for Z, nom in elements.items():
    print(f"{Z:>4} | {nom:>8} | {config_electronique(Z)}")
```

[PIÈGE]

Ne pas confondre Z (numéro atomique = protons) et A (nombre de masse = protons + neutrons). Une erreur fréquente : croire que des isotopes sont des éléments différents – non, ils ont le même Z et donc les mêmes propriétés chimiques.

2. LIAISONS CHIMIQUES

[THÉORIE]

Les atomes s'assemblent en molécules ou en solides grâce aux liaisons chimiques. Ces liaisons résultent de l'interaction entre les électrons de valence. Comprendre les liaisons, c'est comprendre les propriétés physiques (point de fusion, solubilité, conductivité) et chimiques des substances.

[DÉFINITION]

LIAISON COVALENTE :

Partage d'une ou plusieurs paires d'électrons entre deux atomes non métalliques.

Simple (–) : 1 paire partagée ex : H–H, H–Cl

Double (=) : 2 paires partagées ex : O=C=O (CO₂)

Triple (≡) : 3 paires partagées ex : N≡N (N₂)

LIAISON IONIQUE :

Transfert complet d'un ou plusieurs électrons d'un métal vers un non-métal.

Résulte en des ions de charges opposées qui s'attirent.

ex : Na⁺ Cl⁻ → NaCl (sel de table)

LIAISON MÉTALLIQUE :

Dans les métaux, les électrons de valence forment un "nuage" délocalisé autour d'ions positifs. Explique la conductivité et la malléabilité.

ÉLECTRONÉGATIVITÉ (χ) :

Capacité d'un atome à attirer les électrons d'une liaison.

Échelle de Pauling : F=4.0 (max), O=3.5, N=3.0, C=2.5, H=2.1, Na=0.9

$|\Delta\chi| < 0.5 \rightarrow$ covalente apolaire (ex : H_2 , Cl_2)
 $0.5 \leq |\Delta\chi| < 1.7 \rightarrow$ covalente polaire (ex : H_2O , HCl)
 $|\Delta\chi| \geq 1.7 \rightarrow$ ionique (ex : $NaCl$)

LIAISON HYDROGÈNE :

Interaction entre H lié à N, O ou F et un autre atome électronégatif.
Plus faible qu'une liaison covalente, mais cruciale pour l'eau et l'ADN.

[DÉMONSTRATION] Pourquoi l'eau est polaire ?

Molécule H_2O :

O ($\chi=3.5$), H ($\chi=2.1$) $\rightarrow \Delta\chi = 1.4 \rightarrow$ liaisons O-H polaires
La molécule est coudée (angle H-O-H $\approx 104.5^\circ$) à cause des doublets
non liants de O qui repoussent les liaisons.
Les deux liaisons O-H polaires ne se compensent pas (non linéaire)
 $\rightarrow$ la molécule a un moment dipolaire global $\rightarrow$ eau polaire. $\square$

[EXEMPLE]

Molécule : CO_2 (dioxyde de carbone)

C ($\chi=2.5$), O ($\chi=3.5$) $\rightarrow \Delta\chi = 1.0 \rightarrow$ liaisons C=O polaires
Géométrie : linéaire ($O=C=O$) $\rightarrow$ les dipôles se compensent
 $\rightarrow$ molécule non polaire malgré des liaisons polaires !

Molécule : NH_3 (ammoniac)

Géométrie pyramidale $\rightarrow$ dipôles ne se compensent pas $\rightarrow$ polaire

[CODE PYTHON]

```
# Calcul du type de liaison à partir de la différence d'électronégativité

electronegativite = {
    'H': 2.20, 'Li': 0.98, 'C': 2.55, 'N': 3.04, 'O': 3.44,
    'F': 3.98, 'Na': 0.93, 'Mg': 1.31, 'Cl': 3.16, 'K': 0.82,
    'Ca': 1.00, 'Fe': 1.83, 'Br': 2.96, 'I': 2.66
}

def type_liaison(elem1, elem2):
    chi1 = electronegativite.get(elem1)
    chi2 = electronegativite.get(elem2)
    if chi1 is None or chi2 is None:
        return "Élément inconnu"
    delta = abs(chi1 - chi2)
    if delta < 0.5:
        return f"Covalente apolaire  ( $\Delta\chi={delta:.2f}$ )"
    elif delta < 1.7:
        return f"Covalente polaire   ( $\Delta\chi={delta:.2f}$ )"
    else:
        return f"Ionique              ( $\Delta\chi={delta:.2f}$ )"

paires = [('H', 'H'), ('H', 'Cl'), ('H', 'O'), ('Na', 'Cl'), ('C', 'O'), ('N', 'H')]

print(f"{'Paire':>8} | Type de liaison")
print("-" * 45)
for a, b in paires:
    print(f"{a}-{b:>5} | {type_liaison(a, b)}")
```

[PIÈGE]

Une molécule avec des liaisons polaires n'est pas nécessairement polaire.
 CO_2 a des liaisons C=O polaires mais est apolaire (linéaire).
La géométrie compte autant que les électronégativités.

3. STœCHIMÉTRIE ET ÉQUATIONS CHIMIQUES

[THÉORIE]

Une réaction chimique transforme des réactifs en produits. La stœchiométrie (du grec stoikheion = élément, metron = mesure) est la science des quantités en chimie. Elle repose sur la loi de conservation de la masse (Lavoisier) :
"Rien ne se perd, rien ne se crée, tout se transforme."

[DÉFINITION]

MASSE MOLAIRES (M) :

Masse d'une mole d'une substance, en g/mol.
= somme des masses atomiques de tous les atomes de la formule.
ex : $M(\text{H}_2\text{O}) = 2 \times 1 + 16 = 18 \text{ g/mol}$

MOLE (mol) :
Unité de quantité de matière.
1 mole = 6.022×10^{23} entités (nombre d'Avogadro, N_A)

RELATIONS FONDAMENTALES :
 $n = m / M$ (n en mol, m en g, M en g/mol)
 $n = N / N_A$ (N = nombre d'entités)
 $n = C \times V$ (en solution : C en mol/L, V en L)

ÉQUATION CHIMIQUE ÉQUILIBRÉE :
Les coefficients stœchiométriques indiquent les rapports molaires.
ex : $2\text{H}_2 + \text{O}_2 \rightarrow 2\text{H}_2\text{O}$
Signifie : 2 mol de H_2 réagissent avec 1 mol de O_2 pour donner 2 mol de H_2O

RÉACTIF LIMITANT :
Le réactif qui est entièrement consommé en premier et qui limite la quantité de produit formé.

[DÉMONSTRATION] Trouver le réactif limitant
Réaction : $\text{N}_2 + 3\text{H}_2 \rightarrow 2\text{NH}_3$
Données : 28 g de N_2 et 9 g de H_2

$n(\text{N}_2) = 28/28 = 1 \text{ mol}$ ($M_{\text{N}_2} = 2 \times 14 = 28 \text{ g/mol}$)
 $n(\text{H}_2) = 9/2 = 4.5 \text{ mol}$ ($M_{\text{H}_2} = 2 \times 1 = 2 \text{ g/mol}$)

Pour consommer 1 mol de N_2 , il faut 3 mol de H_2 .
On a 4.5 mol de $\text{H}_2 \rightarrow$ on peut en consommer au max $4.5/3 = 1.5 \text{ mol}$ de N_2
Mais on n'a que 1 mol de $\text{N}_2 \rightarrow \text{N}_2$ est le réactif limitant.

Quantité de NH_3 produite :
 $n(\text{NH}_3) = 2 \times n(\text{N}_2) = 2 \times 1 = 2 \text{ mol}$
 $m(\text{NH}_3) = 2 \times 17 = 34 \text{ g}$ ($M_{\text{NH}_3} = 14 + 3 \times 1 = 17 \text{ g/mol}$) □

[EXEMPLE]
Combustion du méthane : $\text{CH}_4 + 2\text{O}_2 \rightarrow \text{CO}_2 + 2\text{H}_2\text{O}$
Si on brûle 16 g de CH_4 (1 mol) :
- On consomme 2 mol d' $\text{O}_2 = 64 \text{ g}$ d' O_2
- On produit 1 mol de $\text{CO}_2 = 44 \text{ g}$ de CO_2
- On produit 2 mol de $\text{H}_2\text{O} = 36 \text{ g}$ d' H_2O
- Vérification masse : $16 + 64 = 80 \text{ g}$ réactifs = $44 + 36 = 80 \text{ g}$ produits ✓

[CODE PYTHON]

```
# Calculateur stœchiométrique

masses_atomiques = {
    'H': 1.008, 'C': 12.011, 'N': 14.007, 'O': 15.999,
    'Na': 22.990, 'Cl': 35.453, 'Ca': 40.078, 'S': 32.06,
    'Fe': 55.845, 'Mg': 24.305, 'K': 39.098, 'P': 30.974
}

def masse_molaire(formule):
    """Calcule la masse molaire d'une formule simple (sans parenthèses)."""
    import re
    elements = re.findall(r'([A-Z][a-z]?)(\d*)', formule)
    M = 0
    for elem, n in elements:
        if elem and elem in masses_atomiques:
            M += masses_atomiques[elem] * (int(n) if n else 1)
    return round(M, 3)

def reactif_limitant(reactifs):
    """
    reactifs = dict {formule: (masse_g, coeff_stoech)}
    Retourne le réactif limitant et la quantité max de produit.
    """
    moles_disponibles = {}
    for formule, (masse, coeff) in reactifs.items():
        M = masse_molaire(formule)
        n = masse / M
        moles_equiv = n / coeff # moles équivalentes (normalisées par coeff)
```

```

    moles_disponibles[formule] = (n, moles_equiv, M)
    print(f" {formule}: m={masse}g, M={M}g/mol, n={n:.4f}mol, "
          f"n_equiv={moles_equiv:.4f}mol")
    limitant = min(moles_disponibles, key=lambda f: moles_disponibles[f][1])
    return limitant, moles_disponibles[limitant][1]

print("=== Synthèse de l'ammoniac : N2 + 3H2 → 2NH3 ===")
reactifs = {
    'N2': (28, 1),    # 28 g, coeff stœchio = 1
    'H2': (9, 3),     # 9 g,  coeff stœchio = 3
}
limitant, n_equiv = reactif_limitant(reactifs)
coeff_NH3 = 2
n_NH3 = n_equiv * coeff_NH3
m_NH3 = n_NH3 * masse_molaire('NH3')
print(f"\nRéactif limitant : {limitant}")
print(f"NH3 produit      : {n_NH3:.4f} mol = {m_NH3:.2f} g")

print("\n=== Masses molaires ===")
for formule in ['H2O', 'NaCl', 'CO2', 'C6H12O6', 'NH3', 'H2SO4']:
    print(f" M({formule}) = {masse_molaire(formule):.3f} g/mol")

```

[PIÈGE]

On calcule la stœchiométrie en MOLES, jamais directement en grammes.
 Erreur classique : "j'ai 10g de A et il faut 2 fois plus de B donc 20g de B."
 FAUX : il faut d'abord convertir en moles en divisant par les masses molaires.

4. THERMODYNAMIQUE CHIMIQUE

[THÉORIE]

Pourquoi certaines réactions se produisent-elles spontanément et d'autres non ?
 La thermodynamique chimique répond à cette question en étudiant les échanges d'énergie. Elle détermine si une réaction est favorable et dans quelle direction elle évolue.

[DÉFINITION]

ENTHALPIE (H) :
 Mesure de l'énergie totale d'un système à pression constante.
 ΔH = enthalpie des produits – enthalpie des réactifs

$\Delta H < 0$: réaction exothermique (dégage de la chaleur)
 $\Delta H > 0$: réaction endothermique (absorbe de la chaleur)

LOI DE HESS :

ΔH d'une réaction = somme des ΔH_f° des produits – somme des ΔH_f° des réactifs
 (ΔH_f° = enthalpie standard de formation d'un composé)

ENTROPIE (S) :

Mesure du désordre ou du nombre de microstates d'un système.
 $\Delta S > 0$: le désordre augmente (favorable)

ÉNERGIE LIBRE DE GIBBS (G) :

$G = H - T \cdot S$ (T en Kelvin)
 $\Delta G = \Delta H - T \cdot \Delta S$

$\Delta G < 0$: réaction spontanée dans le sens direct
 $\Delta G = 0$: équilibre
 $\Delta G > 0$: réaction non spontanée (sens inverse spontané)

[DÉMONSTRATION] Combustion du carbone : $C + O_2 \rightarrow CO_2$

$\Delta H_f^\circ(CO_2) = -393.5$ kJ/mol
 $\Delta H_f^\circ(C) = 0$ kJ/mol (élément pur à l'état standard)
 $\Delta H_f^\circ(O_2) = 0$ kJ/mol

$\Delta H^\circ_{rxn} = \Delta H_f^\circ(CO_2) - \Delta H_f^\circ(C) - \Delta H_f^\circ(O_2)$
 $= -393.5 - 0 - 0 = -393.5$ kJ/mol

$\Delta S^\circ_{rxn} = S^\circ(CO_2) - S^\circ(C) - S^\circ(O_2)$
 $= 213.8 - 5.7 - 205.2 = +2.9$ J/(mol·K)

À T = 298 K :

$\Delta G^\circ = -393500 - 298 \times 2.9 = -393500 - 864 \approx -394.4 \text{ kJ/mol}$
 $\Delta G^\circ \ll 0 \rightarrow$ réaction très spontanée. $\square$

[CODE PYTHON]

```
# Calculateur thermodynamique (loi de Hess)

# Données standard à 298 K (en J/mol et J/mol/K)
donnees_thermo = {
    # Composé : (ΔHf° en J/mol, S° en J/mol/K)
    'H2O_l': (-285830, 69.95),
    'H2O_g': (-241818, 188.72),
    'CO2': (-393500, 213.79),
    'O2': (0, 205.20),
    'H2': (0, 130.68),
    'C': (0, 5.74),
    'CH4': (-74870, 186.25),
    'N2': (0, 191.61),
    'NH3': (-46110, 192.77),
    'NaCl_s': (-411200, 72.11),
    'Na_s': (0, 51.30),
    'Cl2_g': (0, 223.08),
}

def gibbs(reaction, T=298):
    """
    reaction = dict {composé: coeff}
    coeff > 0 → produit
    coeff < 0 → réactif
    T = température en Kelvin
    """
    dH = sum(coeff * donnees_thermo[comp][0]
              for comp, coeff in reaction.items())
    dS = sum(coeff * donnees_thermo[comp][1]
              for comp, coeff in reaction.items())
    dG = dH - T * dS
    return dH, dS, dG

# Combustion du méthane : CH4 + 2O2 → CO2 + 2H2O(g)
rxn_methane = {'CO2': 1, 'H2O_g': 2, 'CH4': -1, 'O2': -2}
dH, dS, dG = gibbs(rxn_methane)
print("=== Combustion du méthane : CH4 + 2O2 → CO2 + 2H2O(g) ===")
print(f"ΔH° = {dH/1000:+.1f} kJ/mol")
print(f"ΔS° = {dS:+.2f} J/(mol·K)")
print(f"ΔG° (298K) = {dG/1000:+.1f} kJ/mol")
print(f"Spontanée ? {'Oui' if dG < 0 else 'Non'}")

# Variation de spontanéité avec T : synthèse de l'eau
print("\n=== Synthèse de l'eau : H2 + ½O2 → H2O(g) ===")
rxn_eau = {'H2O_g': 1, 'H2': -1, 'O2': -0.5}
print(f"{T (K):>8} | {'ΔG (kJ/mol)':>12} | Spontanée ?")
print("-" * 38)
for T in [298, 500, 1000, 2000, 3000]:
    dH, dS, dG = gibbs(rxn_eau, T)
    print(f"{T:>8} | {dG/1000:>+12.1f} | {'Oui' if dG < 0 else 'Non'}")
```

[PIÈGE]

$\Delta G < 0$ signifie que la réaction est THERMODYNAMIQUEMENT favorable (spontanée), pas qu'elle sera RAPIDE. Un diamant se transforme spontanément en graphite ($\Delta G < 0$) mais si lentement que cela n'arrive jamais à l'échelle humaine. La cinétique et la thermodynamique sont deux choses différentes.

5. CINÉTIQUE CHIMIQUE ET ÉQUILIBRES

[THÉORIE]

La thermodynamique dit si une réaction peut se produire ; la cinétique dit à quelle vitesse. De plus, la plupart des réactions chimiques n'arrivent pas à leur terme : elles atteignent un ÉQUILIBRE dynamique où les réactions directe et inverse se font à la même vitesse.

[DÉFINITION]

VITESSE DE RÉACTION (v) :
 $v = -(1/a) \cdot d[A]/dt = (1/b) \cdot d[B]/dt$ (pour $aA \rightarrow bB$)
Unité : $\text{mol} \cdot \text{L}^{-1} \cdot \text{s}^{-1}$

LOI DE VITESSE :
 $v = k \cdot [A]^m \cdot [B]^n$
k = constante de vitesse (dépend de T)
m, n = ordres partiels (déterminés expérimentalement)

ORDRE GLOBAL = m + n

LOI D'ARRHENIUS :
 $k = A \cdot e^{(-E_a/RT)}$
 E_a = énergie d'activation (en J/mol)
R = 8.314 J/(mol·K)
T = température en Kelvin
→ la vitesse augmente exponentiellement avec T

CONSTANTE D'ÉQUILIBRE (K) :
Pour $aA + bB \rightleftharpoons cC + dD$:
 $K = [C]^c \cdot [D]^d / ([A]^a \cdot [B]^b)$ (concentrations à l'équilibre)

K >> 1 : équilibre déplacé vers les produits
K << 1 : équilibre déplacé vers les réactifs
K = 1 : mélange des deux

PRINCIPE DE LE CHATELIER :
Si on perturbe un système en équilibre (concentration, pression, T),
il évolue dans le sens qui diminue la perturbation.

[DÉMONSTRATION] Loi d'ordre 1 : intégration

Pour une réaction d'ordre 1 : $v = k \cdot [A]$

$$\begin{aligned} d[A]/dt &= -k \cdot [A] \\ d[A]/[A] &= -k \cdot dt \\ \int d[A]/[A] &= -k \int dt \\ \ln[A] &= -k \cdot t + \text{cste} \\ [A](t) &= [A]_0 \cdot e^{(-kt)} \end{aligned}$$

DEMI-VIE ($t_{1/2}$) : temps pour que [A] diminue de moitié
 $[A]_0/2 = [A]_0 \cdot e^{(-k \cdot t_{1/2})}$
 $1/2 = e^{(-k \cdot t_{1/2})}$
 $\ln(1/2) = -k \cdot t_{1/2}$
 $t_{1/2} = \ln(2)/k \approx 0.693/k$ (indépendant de $[A]_0$!) □

[CODE PYTHON]

```
import math

# Cinétique d'ordre 1 : [A](t) = [A]0 * exp(-k*t)
def concentration_ordrel(A0, k, t):
    return A0 * math.exp(-k * t)

def demi_vie_ordrel(k):
    return math.log(2) / k

# Exemple : décomposition de N2O5 (k = 0.0035 s⁻¹ à 25°C)
k = 0.0035 # s⁻¹
A0 = 1.0 # mol/L initial

print("=== Décomposition N2O5 (ordre 1, k=0.0035 s⁻¹) ===")
print(f"Demi-vie : {demi_vie_ordrel(k):.1f} s")
print(f"\n{'t (s)':>8} | {'[A] (mol/L)':>12} | {'% restant':>10}")
print("-" * 38)
for t in [0, 100, 200, 300, 400, 500, 600]:
    A = concentration_ordrel(A0, k, t)
    print(f"{'t':>8} | {'A:>12.4f'} | {'A/A0*100:>9.1f}%")

# Loi d'Arrhenius : variation de k avec T
R = 8.314 # J/(mol·K)
A_freq = 1e13 # facteur pré-exponentiel
Ea = 75000 # énergie d'activation en J/mol (75 kJ/mol)

print("\n=== Loi d'Arrhenius (Ea=75 kJ/mol) ===")
print(f"{'T (°C)':>8} | {'T (K)':>6} | {'k relatif':>12}")
print("-" * 35)
```

```

k_ref = A_freq * math.exp(-Ea / (R * 298))
for T_celsius in [0, 10, 25, 37, 50, 75, 100]:
    T = T_celsius + 273.15
    k = A_freq * math.exp(-Ea / (R * T))
    print(f"{T_celsius:>8} | {T:>6.1f} | {k/k_ref:>12.2f}")

```

[PIÈGE]

L'ordre d'une réaction n'est PAS donné par les coefficients stœchiométriques (sauf pour les réactions élémentaires). Il est TOUJOURS déterminé expérimentalement. Écrire $v = k[A]^2[B]$ pour $A + 2B \rightarrow C$ est une erreur si ce n'est pas la réaction élémentaire.

6. ACIDES, BASES ET pH

[THÉORIE]

La chimie acide-base est omniprésente : dans le corps humain (pH sanguin 7.35-7.45), dans l'industrie, dans la cuisine. Comprendre ce concept est fondamental en chimie et en biologie.

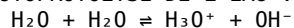
[DÉFINITION]

DÉFINITION DE BRØNSTED-LOWRY :

Acide = donneur de proton (H^+)

Base = accepteur de proton (H^+)

AUTOPROTOLYSE DE L'EAU :



$$K_e = [H_3O^+] \cdot [OH^-] = 10^{-14} \quad (\text{à } 25^\circ C)$$

$$\rightarrow pK_e = 14$$

$$pH = -\log_{10}[H_3O^+]$$

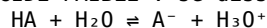
$$pOH = -\log_{10}[OH^-]$$

$$pH + pOH = 14 \quad (\text{à } 25^\circ C)$$

ACIDE FORT : se dissocie complètement (HCl , HNO_3 , H_2SO_4 ...)

$$[H_3O^+] = Ca \rightarrow pH = -\log(Ca)$$

ACIDE FAIBLE : se dissocie partiellement



$$K_a = \frac{[A^-][H_3O^+]}{[HA]}$$

$$pK_a = -\log(K_a)$$

$$pH \approx \frac{1}{2}(pK_a - \log Ca) \quad (\text{approximation valide si } K_a \ll Ca)$$

TAMPON :

Solution d'un acide faible et de sa base conjuguée.

Équation de Henderson-Hasselbalch :

$$pH = pK_a + \log\left(\frac{[A^-]}{[HA]}\right)$$

$\rightarrow$ résiste aux variations de pH

[EXEMPLE]

Acide acétique (vinaigre) : CH_3COOH , $pK_a = 4.76$

Solution à 0.1 mol/L :

$$pH = \frac{1}{2}(4.76 - \log(0.1)) = \frac{1}{2}(4.76 + 1) = \frac{1}{2} \times 5.76 = 2.88$$

Tampon acétate $pH = 5.0$:

$$pH = pK_a + \log\left(\frac{[CH_3COO^-]}{[CH_3COOH]}\right)$$

$$5.0 = 4.76 + \log(r)$$

$$\log(r) = 0.24$$

$$r = 10^{0.24} \approx 1.74$$

$$\rightarrow \text{rapport base/acide} \approx 1.74$$

[CODE PYTHON]

```
import math
```

```
# Calculateur de pH
```

```
def pH_acide_fort(concentration):
    """pH d'un acide fort de concentration C (mol/L)."""
    if concentration <= 0:
```



```

    return None
    return -math.log10(concentration)

def pH_base_forte(concentration):
    """pH d'une base forte de concentration C (mol/L)."""
    pOH = -math.log10(concentration)
    return 14 - pOH

def pH_acide_faible(Ca, pKa):
    """pH d'un acide faible (approximation)."""
    Ka = 10**(-pKa)
    # Résolution exacte : Ka = x²/(Ca-x), x = [H₃O⁺]
    # x² + Ka*x - Ka*Ca = 0
    x = (-Ka + math.sqrt(Ka**2 + 4*Ka*Ca)) / 2
    return -math.log10(x)

def pH_tampon(pKa, C_acide, C_base):
    """pH d'un tampon (Henderson-Hasselbalch)."""
    return pKa + math.log10(C_base / C_acide)

# Quelques acides courants (pKa)
acides = {
    'HCl (fort)': ('fort', None),
    'H₂SO₄ (fort)': ('fort', None),
    'H₃PO₄': ('faible', 2.15),
    'HF': ('faible', 3.17),
    'CH₃COOH (vinaigre)': ('faible', 4.76),
    'H₂CO₃': ('faible', 6.35),
    'H₂O': ('faible', 15.74),
}

print("=== pH de solutions 0.1 mol/L ===")
print(f"{'Acide':>25} | {'pH':>6}")
print("-" * 35)
for nom, (type_, pKa) in acides.items():
    if type_ == 'fort':
        pH = pH_acide_forte(0.1)
    else:
        pH = pH_acide_faible(0.1, pKa)
    print(f"{'nom':>25} | {'pH':>6.2f}")

print("\n=== Courbe de titrage HCl (acide fort) vs NaOH (base forte) ===")
print("Volume NaOH ajouté (en fraction du volume équivalent) | pH")
print("-" * 55)
Ca = 0.1 # mol/L d'acide
Va = 50 # mL d'acide
Cb = 0.1 # mol/L de base

for frac in [0, 0.25, 0.5, 0.75, 0.9, 0.99, 1.0, 1.01, 1.1, 1.5, 2.0]:
    Vb = frac * Va # mL de base
    n_acide_init = Ca * Va / 1000
    n_base = Cb * Vb / 1000
    V_total = (Va + Vb) / 1000
    n_HCl_restant = n_acide_init - n_base
    if abs(n_HCl_restant) < 1e-10:
        pH = 7.0
    elif n_HCl_restant > 0:
        pH = -math.log10(n_HCl_restant / V_total)
    else: # excès de base
        n_OH = -n_HCl_restant
        pOH = -math.log10(n_OH / V_total)
        pH = 14 - pOH
    print(f" f={frac:.2f} (Vb={Vb:5.1f} mL) | pH = {pH:5.2f}")

```

[PIÈGE]

Le pH d'un acide faible n'est pas simplement $-\log(\text{Ca})$!
 Exemple : HF à 0.1 mol/L → pH ≈ 2.08, et non 1.0 comme pour HCl.
 Il faut résoudre l'équation de Ka. L'approximation $\text{pH} = \frac{1}{2}(\text{pKa} - \log \text{Ca})$
 n'est valide que si le taux de dissociation est faible (< 5%).

7. LA CELLULE – UNITÉ DU VIVANT

[THÉORIE]

La cellule est la plus petite unité structurelle et fonctionnelle du vivant. Tous les organismes sont composés d'une ou plusieurs cellules (théorie cellulaire, Schleiden & Schwann, 1838-39). Il existe deux grands types : les cellules procaryotes (sans noyau) et les cellules eucaryotes (avec noyau).

[DÉFINITION]

CELLULE PROCARYOTE (bactéries, archées) :

- Pas de noyau délimité par une membrane
- ADN circulaire dans le nucléoïde
- Taille : 1–10 μm
- Pas d'organites membranaires
- Ribosomes 70S

CELLULE EUCARYOTE (animaux, plantes, champignons, protistes) :

- Noyau avec membrane nucléaire
- ADN linéaire dans des chromosomes
- Taille : 10–100 μm
- Nombreux organites membranaires
- Ribosomes 80S

ORGANITES MAJEURS (eucaryotes) :

- Noyau : contient l'ADN, siège de la transcription
- Mitochondrie : production d'ATP (respiration cellulaire)
- Réticulum endoplasmique (RE) :
 - RE rugueux : synthèse et modification des protéines
 - RE lisse : synthèse des lipides, détoxification
- Appareil de Golgi : tri et export des protéines
- Lysosome : digestion intracellulaire (enzymes hydrolytiques)
- Ribosome : synthèse des protéines (traduction)
- Chloroplaste : photosynthèse (cellules végétales seulement)
- Vacuole centrale : stockage, turgescence (cellules végétales)
- Centrosome : organisation du fuseau mitotique

MEMBRANE PLASMIQUE :

- Bicouche lipidique (phospholipides) + protéines (modèle mosaïque fluide)
- Rôles : barrière sélective, signalisation, transport

[EXEMPLE]

- Une bactérie E. coli mesure $\sim 2 \mu\text{m} \times 0.5 \mu\text{m}$.
- Un globule rouge humain mesure $\sim 8 \mu\text{m}$ de diamètre.
- Un neurone humain peut mesurer jusqu'à 1 mètre (axone) !
- Un ovocyte humain est visible à l'œil nu : $\sim 120 \mu\text{m}$ de diamètre.

[CODE PYTHON]

```
# Comparaison de tailles cellulaires et calculs volumiques

import math

cellules = {
    'Virus (VIH)':          0.12e-6,  # 120 nm
    'Ribosome':             0.025e-6, # 25 nm
    'E. coli (bactérie)':    2e-6,
    'Globule rouge':         8e-6,
    'Cellule hépatique':     20e-6,
    'Ovocyte humain':       120e-6,
    'Neurone (corps)':       50e-6,
}

print(f"{'Cellule/organite':>22} | {'Diamètre (μm)':>14} | {'Volume (fL)':>12}")
print("-" * 56)
for nom, d in cellules.items():
    r = d / 2
    V_m3 = (4/3) * math.pi * r**3          # m³ (sphère)
    V_fL = V_m3 * 1e18                     # femtolitres (1 fL = 10-15 L)
    print(f"{'nom':>22} | {'d*1e6':>14.2f} | {'V_fL':>12.4g}")

# Proportion du noyau dans la cellule (règle de 10% du diamètre)
print("\n=== Noyau vs Cellule ===")
```

```
d_cellule = 20e-6 # cellule typique 20 µm
d_noyau = 6e-6 # noyau typique 6 µm
ratio_vol = (d_noyau / d_cellule)**3 * 100
print(f"Diamètre cellule : {d_cellule*1e6:.0f} µm")
print(f"Diamètre noyau : {d_noyau*1e6:.0f} µm")
print(f"Volume noyau / cellule ≈ {ratio_vol:.1f}%")
```

[PIÈGE]

Les ribosomes ne sont PAS des organites membranaires – ils n'ont pas de membrane. Ils existent dans toutes les cellules (70S chez les procaryotes, 80S chez les eucaryotes). Les antibiotiques comme la streptomycine ciblent spécifiquement les ribosomes 70S, épargnant les cellules humaines (80S).

8. ADN, ARN ET EXPRESSION GÉNÉTIQUE

[THÉORIE]

Le dogme central de la biologie moléculaire (Crick, 1958) décrit le flux d'information génétique : ADN → ARN → Protéine. Cette information codée dans la séquence de bases azotées est transcrite puis traduite pour produire les protéines qui font fonctionner la cellule.

[DÉFINITION]

ADN (acide désoxyribonucléique) :

Double hélice antiparallèle de deux brins complémentaires.

Bases azotées : A (adénine), T (thymine), G (guanine), C (cytosine)

Appariements : A=T (2 liaisons H), G≡C (3 liaisons H)

Sucre : désoxyribose

ARN (acide ribonucléique) :

Généralement simple brin.

Bases : A, U (uracile), G, C (U remplace T)

Sucre : ribose

TYPES D'ARN :

ARNm (messenger) : porte l'information du gène au ribosome

ARNt (transfert) : apporte les acides aminés au ribosome

ARNr (ribosomique) : composant structural des ribosomes

TRANSCRIPTION :

ADN → ARNm (dans le noyau)

L'ARN polymérase lit le brin matrice 3'→5' et synthétise l'ARNm 5'→3'

TRADUCTION :

ARNm → Protéine (au ribosome)

CODON : triplet de nucléotides sur l'ARNm codant pour un acide aminé

64 codons possibles (4³) pour 20 acides aminés → code dégénéré

Codon initiateur : AUG (méthionine)

Codons stop : UAA, UAG, UGA

COMPLÉMENTARITÉ DES BASES :

ADN : A↔T, G↔C

ADN/ARN : A↔U, T↔A, G↔C, C↔G

[DÉMONSTRATION] Du gène à la protéine

Brin matrice ADN (3'→5') : 3' - TAC - GGA - CTT - ATG - 5'

Transcription (ARNm, 5'→3') :

Brin matrice 3'→5' : T A C G G A C T T A T G

ARNm 5'→3' : A U G C C U G A A U A C

Traduction (codons) :

AUG → Méthionine (Met, M) [initiateur]

CCU → Proline (Pro, P)

GAA → Acide glutamique (Glu, E)

UAC → Tyrosine (Tyr, Y)

Séquence protéique : Met-Pro-Glu-Tyr (M-P-E-Y) □

[CODE PYTHON]

Simulateur du dogme central : ADN → ARN → Protéine

```

# Table du code génétique (codons ARNm → acide aminé)
code_genetique = {
    'UUU': 'Phe', 'UUC': 'Phe', 'UUA': 'Leu', 'UUG': 'Leu',
    'CUU': 'Leu', 'CUC': 'Leu', 'CUA': 'Leu', 'CUG': 'Leu',
    'AUU': 'Ile', 'AUC': 'Ile', 'AUA': 'Ile', 'AUG': 'Met*',
    'GUU': 'Val', 'GUC': 'Val', 'GUA': 'Val', 'GUG': 'Val',
    'UCU': 'Ser', 'UCC': 'Ser', 'UCA': 'Ser', 'UCG': 'Ser',
    'CCU': 'Pro', 'CCC': 'Pro', 'CCA': 'Pro', 'CCG': 'Pro',
    'ACU': 'Thr', 'ACC': 'Thr', 'ACA': 'Thr', 'ACG': 'Thr',
    'GCU': 'Ala', 'GCC': 'Ala', 'GCA': 'Ala', 'GCG': 'Ala',
    'UAU': 'Tyr', 'UAC': 'Tyr', 'UAA': 'STOP', 'UAG': 'STOP',
    'CAU': 'His', 'CAC': 'His', 'CAA': 'Gln', 'CAG': 'Gln',
    'AAU': 'Asn', 'AAC': 'Asn', 'AAA': 'Lys', 'AAG': 'Lys',
    'GAU': 'Asp', 'GAC': 'Asp', 'GAA': 'Glu', 'GAG': 'Glu',
    'UGU': 'Cys', 'UGC': 'Cys', 'UGA': 'STOP', 'UGG': 'Trp',
    'CGU': 'Arg', 'CGC': 'Arg', 'CGA': 'Arg', 'CGG': 'Arg',
    'AGU': 'Ser', 'AGC': 'Ser', 'AGA': 'Arg', 'AGG': 'Arg',
    'GGU': 'Gly', 'GGC': 'Gly', 'GGA': 'Gly', 'GGG': 'Gly',
}

def complementaire_ADN(brin):
    """Brin complémentaire d'ADN (antiparallèle)."""
    comp = {'A': 'T', 'T': 'A', 'G': 'C', 'C': 'G'}
    return ''.join(comp.get(b, '?') for b in reversed(brin.upper()))

def transcrire(brin_matrice_3_5):
    """ADN brin matrice (3'→5') → ARNm (5'→3')."""
    comp_ARN = {'A': 'U', 'T': 'A', 'G': 'C', 'C': 'G'}
    return ''.join(comp_ARN.get(b, '?') for b in brin_matrice_3_5.upper())

def traduire(ARNm):
    """ARNm (5'→3') → séquence protéique."""
    proteine = []
    ARNm = ARNm.upper().replace(' ', '')
    # Chercher AUG
    start = ARNm.find('AUG')
    if start == -1:
        return "Pas de codon AUG trouvé"
    for i in range(start, len(ARNm) - 2, 3):
        codon = ARNm[i:i+3]
        if len(codon) < 3:
            break
        aa = code_genetique.get(codon, '?')
        if aa == 'STOP':
            break
        proteine.append(aa)
    return '-'.join(proteine)

# Exemple complet
brin_matrice = "TACGGACTTATG" # 3'→5'
print(f"Brin matrice ADN (3'→5') : {brin_matrice}")

ARNm = transcrire(brin_matrice)
print(f"ARNm (5'→3') : {ARNm}")

proteine = traduire(ARNm)
print(f"Protéine : {proteine}")

# Affichage des codons
print("\nDécodage codon par codon :")
start = ARNm.find('AUG')
for i in range(start, len(ARNm) - 2, 3):
    codon = ARNm[i:i+3]
    if len(codon) < 3:
        break
    aa = code_genetique.get(codon, '?')
    print(f" {codon} → {aa}")
    if aa == 'STOP':
        break

```

[PIÈGE]

La transcription lit le brin MATRICE (3'→5') et produit l'ARNm (5'→3').
Le brin codant (ou sens) de l'ADN a la même séquence que l'ARNm, mais

avec T à la place de U. Ne pas confondre les deux brins est crucial.
De plus, les gènes eucaryotes contiennent des INTRONS (non codants) qui sont éliminés lors de l'épissage avant la traduction.

9. ENZYMES ET CINÉTIQUE ENZYMATIQUE

[THÉORIE]

Les enzymes sont des catalyseurs biologiques (généralement des protéines) qui accélèrent les réactions chimiques du vivant sans être consommées. Sans elles, les réactions du métabolisme seraient beaucoup trop lentes pour entretenir la vie. Une enzyme peut accélérer une réaction d'un facteur 10^6 à 10^{12} .

[DÉFINITION]

SUBSTRAT (S) : molécule sur laquelle agit l'enzyme
PRODUIT (P) : molécule résultant de la réaction
SITE ACTIF : poche de l'enzyme où se lie le substrat

MODÈLE MICHAELIS-MENTEN :



ÉQUATION DE MICHAELIS-MENTEN :

$$v = V_{\max} \cdot [S] / (K_m + [S])$$

$V_{\max}$ = vitesse maximale (quand $[S] \rightarrow \infty$, tous les sites actifs occupés)
 K_m = constante de Michaelis = $[S]$ pour lequel $v = V_{\max}/2$
→ mesure l'affinité enzyme-substrat (K_m faible = haute affinité)

INHIBITION :

Compétitive : l'inhibiteur se lie au site actif (augmente K_m apparent)
Non compétitive : l'inhibiteur se lie ailleurs (diminue $V_{\max}$ apparent)
Incompétitive : se lie à ES uniquement

FACTEURS AFFECTANT L'ACTIVITÉ ENZYMATIQUE :

- Température (optimum $\sim 37^\circ\text{C}$ chez l'homme, dénaturation au-delà)
- pH (chaque enzyme a un pH optimal)
- Concentration en substrat
- Cofacteurs et coenzymes (vitamines, ions métalliques)
- Inhibiteurs ou activateurs

[DÉMONSTRATION] Détermination de K_m et $V_{\max}$ (méthode de Lineweaver-Burk)

L'équation de Michaelis-Menten peut être linéarisée :

$$1/v = (K_m/V_{\max}) \cdot (1/[S]) + 1/V_{\max}$$

En traçant $1/v$ en fonction de $1/[S]$ (double inverse) :

Pente = $K_m / V_{\max}$
Ordonnée à l'origine = $1/V_{\max}$
Abscisse à l'origine = $-1/K_m$

Ce graphe (Lineweaver-Burk) permet de déterminer graphiquement K_m et $V_{\max}$ à partir de données expérimentales. □

[CODE PYTHON]

Cinétique enzymatique Michaelis-Menten

```
def michaelis_menten(S, Vmax, Km):
    """Vitesse enzymatique en fonction de la concentration en substrat."""
    return Vmax * S / (Km + S)

# Paramètres de la lactate déshydrogénase (exemple)
Vmax = 100 # µmol/(min·mg)
Km = 0.5 # mmol/L

print("=== Courbe v = f([S]) – Michaelis-Menten ===")
print(f"Vmax = {Vmax}, Km = {Km} mmol/L")
print(f"\n{[S] (mM):>10} | {v (µmol/min):>13} | {'% Vmax':>8}")
print("-" * 40)
for S in [0.05, 0.1, 0.25, 0.5, 1.0, 2.0, 5.0, 10.0, 50.0]:
    v = michaelis_menten(S, Vmax, Km)
    print(f"{S:>10.2f} | {v:>13.2f} | {v/Vmax*100:>7.1f}%")
```

```

# Inhibition compétitive : Km apparent augmente, Vmax inchangé
print("\n=== Inhibition compétitive ===")
Ki = 0.2      # constante d'inhibition
I = 0.5      # concentration de l'inhibiteur

alpha = 1 + I / Ki      # facteur d'inhibition
Km_app = Km * alpha     # Km apparent

print(f"Ki={Ki}, [I]={I} → α={alpha:.2f}, Km_apparent={Km_app:.2f} mM")
print(f"\n{'[S] (mM)':>10} | {'v sans inh':>11} | {'v avec inh':>11}")
print("-" * 38)
for S in [0.1, 0.5, 1.0, 2.0, 5.0, 10.0]:
    v_normal = michaelis_menten(S, Vmax, Km)
    v_inhibe = michaelis_menten(S, Vmax, Km_app) # Vmax inchangé
    print(f"{S:>10.1f} | {v_normal:>11.2f} | {v_inhibe:>11.2f}")

# Linéarisation de Lineweaver-Burk
print("\n=== Graphe de Lineweaver-Burk (double inverse) ===")
print(f"{'1/[S]':>8} | {'1/v':>8}")
print("-" * 20)
for S in [0.25, 0.5, 1.0, 2.0, 5.0, 10.0]:
    v = michaelis_menten(S, Vmax, Km)
    print(f"{'1/S':>8.3f} | {'1/v':>8.5f}")
print(f"\nPente = Km/Vmax = {Km/Vmax:.4f}")
print(f"Ordonnée à l'origine = 1/Vmax = {1/Vmax:.4f}")
print(f"Abscisse à l'origine = -1/Km = {-1/Km:.2f}")

```

[PIÈGE]

Le Km n'est PAS une constante d'équilibre simple. C'est la concentration de substrat pour laquelle la vitesse est la moitié de Vmax. Un Km faible signifie une FORTE affinité (l'enzyme est saturée à faible [S]).
 Ne pas confondre Km et Vmax : inhibition compétitive → Km change mais Vmax reste constant. Inhibition non compétitive → Vmax change mais Km reste.

10. MÉTABOLISME – RESPIRATION ET PHOTOSYNTHÈSE

[THÉORIE]

Le métabolisme désigne l'ensemble des réactions chimiques qui se produisent dans une cellule pour maintenir la vie. Le catabolisme dégrade les molécules pour libérer de l'énergie (ATP). L'anabolisme construit des molécules complexes en consommant de l'énergie. L'ATP est la "monnaie énergétique" universelle du vivant.

[DÉFINITION]

ATP (adénosine triphosphate) :
 L'énergie est stockée dans la liaison phosphate anhydride.
 $\text{ATP} + \text{H}_2\text{O} \rightarrow \text{ADP} + \text{P}_i$ ($\Delta G^\circ = -30.5 \text{ kJ/mol}$)

RESPIRATION CELLULAIRE (aérobie) :
 $\text{C}_6\text{H}_{12}\text{O}_6 + 6\text{O}_2 \rightarrow 6\text{CO}_2 + 6\text{H}_2\text{O}$ ($\Delta G^\circ = -2870 \text{ kJ/mol}$)
 Rendement théorique : ~30-32 ATP par glucose

3 étapes :

1. GLYCOLYSE (cytoplasme) : glucose → 2 pyruvates + 2 ATP + 2 NADH
2. CYCLE DE KREBS (matrice mitochondriale) :
 $\text{pyruvate} \rightarrow \text{CO}_2 + \text{NADH} + \text{FADH}_2 + 1 \text{ ATP}$
3. CHAÎNE DE TRANSPORT D'ÉLECTRONS (crêtes mitochondriales) :
 $\text{NADH} + \text{FADH}_2 \rightarrow \text{H}_2\text{O} + \sim 28 \text{ ATP}$ (via ATP synthase, gradient de protons)

PHOTOSYNTHÈSE :

$6\text{CO}_2 + 6\text{H}_2\text{O} + \text{lumière} \rightarrow \text{C}_6\text{H}_{12}\text{O}_6 + 6\text{O}_2$
 Se déroule dans les chloroplastes.

2 étapes :

- Phase lumineuse (thylakoïdes) :
 $\text{Lumière} \rightarrow \text{ATP} + \text{NADPH} + \text{O}_2$ (photolyse de l'eau)
- Cycle de Calvin (stroma) :
 $\text{CO}_2 + \text{ATP} + \text{NADPH} \rightarrow \text{glucose}$ (fixation du CO_2)

COEFFICIENT RESPIRATOIRE (RQ) :

RQ = mol CO₂ produit / mol O₂ consommé
 Glucides : RQ = 1.0
 Lipides : RQ ≈ 0.7
 Protéines: RQ ≈ 0.8

[CODE PYTHON]

```
# Bilan énergétique de la respiration cellulaire

def bilan_respiration(mol_glucose):
    """Calcule le bilan ATP de la respiration aérobie."""
    # Par mole de glucose :
    ATP_glycolyse = 2 * mol_glucose # net
    NADH_glycolyse = 2 * mol_glucose # → ~2.5 ATP chacun via CTE
    pyruvate = 2 * mol_glucose

    # Décarboxylation oxydative du pyruvate (2× par glucose)
    NADH_pyruv = 2 * mol_glucose # → ~2.5 ATP chacun

    # Cycle de Krebs (2× par glucose)
    ATP_krebs = 2 * mol_glucose # GTP = ATP équivalent
    NADH_krebs = 6 * mol_glucose # → ~2.5 ATP chacun
    FADH2_krebs = 2 * mol_glucose # → ~1.5 ATP chacun

    # Bilan ATP (rendement moderne)
    ATP_total = (ATP_glycolyse
                 + NADH_glycolyse * 2.5
                 + NADH_pyruv * 2.5
                 + ATP_krebs
                 + NADH_krebs * 2.5
                 + FADH2_krebs * 1.5)

    return {
        'ATP glycolyse': ATP_glycolyse,
        'ATP (NADH glycolyse → CTE)': NADH_glycolyse * 2.5,
        'ATP (NADH pyruvate → CTE)': NADH_pyruv * 2.5,
        'ATP cycle de Krebs': ATP_krebs,
        'ATP (NADH Krebs → CTE)': NADH_krebs * 2.5,
        'ATP (FADH2 Krebs → CTE)': FADH2_krebs * 1.5,
        'TOTAL ATP': ATP_total
    }

print("=== Bilan de la respiration aérobie (1 mol de glucose) ===")
bilan = bilan_respiration(1)
for etape, ATP in bilan.items():
    if etape == 'TOTAL ATP':
        print("-" * 45)
        print(f" {etape:40} : {ATP:.1f} ATP")

# Efficacité vs combustion directe
dG_combustion = 2870 # kJ/mol
dG_ATP = 30.5 # kJ/mol
ATP_produits = bilan['TOTAL ATP']
efficacite = (ATP_produits * dG_ATP / dG_combustion) * 100

print(f"\nÉnergie totale de combustion du glucose : {dG_combustion} kJ/mol")
print(f"Énergie capturée en ATP : {ATP_produits:.1f} × {dG_ATP} = "
      f"{ATP_produits*dG_ATP:.0f} kJ/mol")
print(f"Efficacité énergétique : {efficacite:.1f}%")

# Photosynthèse : bilan
print("\n=== Bilan photosynthèse (fixation 6 CO2) ===")
print("Phase lumineuse :")
print(" 12 H2O + lumière → 24 H+ + 24 e- + 6 O2")
print(" 18 ATP + 12 NADPH produits")
print("Cycle de Calvin :")
print(" 6 CO2 + 18 ATP + 12 NADPH → 1 glucose + 18 ADP + 12 NADP+")
```

[PIÈGE]

On entend souvent "38 ATP par glucose" – c'est l'ancienne valeur théorique.
 La valeur moderne est ~30-32 ATP, car les transporteurs mitochondriaux
 (navettes pour le NADH cytoplasmique) ont un coût énergétique. La photosynthèse
 ne produit pas de glucides dans les thylakoïdes : c'est dans le stroma
 (cycle de Calvin) que le CO₂ est fixé.

11. GÉNÉTIQUE – LOIS DE MENDEL ET HÉRÉDITÉ

[THÉORIE]

Gregor Mendel (1865) a découvert les lois de l'hérédité en croisant des pois. Ses lois expliquent comment les traits sont transmis des parents aux enfants via des unités héréditaires (que nous appelons aujourd'hui gènes).

[DÉFINITION]

GÈNE : segment d'ADN codant pour un trait héréditaire
ALLÈLE : version d'un gène (ex : allèle A ou a pour la couleur des fleurs)
LOCUS : position d'un gène sur un chromosome
GÉNOTYPE : constitution allélique (ex : AA, Aa, aa)
PHÉNOTYPE : trait observable résultant du génotype + environnement

DOMINANCE :

Dominant (A) : s'exprime même en un seul exemplaire (AA ou Aa)
Récessif (a) : ne s'exprime que si présent en deux exemplaires (aa)
Codominant : les deux allèles s'expriment simultanément

HOMOZYGOTE : deux allèles identiques (AA ou aa)

HÉTÉROZYGOTE : deux allèles différents (Aa)

LOI 1 DE MENDEL – SÉGRÉGATION :

Lors de la gamétogenèse, les deux allèles d'un gène se séparent.
Chaque gamète ne reçoit qu'un allèle de chaque paire.

LOI 2 DE MENDEL – ASSORTIMENT INDÉPENDANT :

Les allèles de gènes situés sur des chromosomes différents sont transmis indépendamment les uns des autres.

CARRÉ DE PUNNETT :

Outil graphique pour prédire les probabilités génotypiques et phénotypiques d'un croisement.

[EXEMPLE]

Croisement Aa × Aa (hybrides pour un caractère) :

	A	a
A	AA	Aa
a	Aa	aa

Génotypes : AA (25%), Aa (50%), aa (25%) → ratio 1:2:1

Phénotypes (A dominant) : A_ (75%), aa (25%) → ratio 3:1

[CODE PYTHON]

```
# Génétique mendélienne et carré de Punnett
```

```
from itertools import product
```

```
def gametes(genotype):
```

```
    """Génère tous les gamètes possibles d'un génotype (1 ou 2 gènes)."""
```

```
    gènes = [genotype[i:i+2] for i in range(0, len(genotype), 2)]
```

```
    combinaisons = list(product(*gènes))
```

```
    return [''.join(c) for c in combinaisons]
```

```
def punnett(parent1, parent2):
```

```
    """Croisement entre deux parents, retourne les proportions génotypiques."""
```

```
    g1 = gametes(parent1)
```

```
    g2 = gametes(parent2)
```

```
    descendants = {}
```

```
    for ga, gb in product(g1, g2):
```

```
        # Trier les allèles pour normaliser (AA = Aa ≠ aa)
```

```
        descendant = ''.join(sorted(ga + gb, key=lambda x: x.lower() + x))
```

```
        # Réordonner : majuscules d'abord pour chaque gène
```

```
        gènes_desc = [descendant[i:i+2] for i in range(0, len(descendant), 2)]
```

```
        gènes_ordo = [''.join(sorted(g, key=lambda x: (x.islower(), x)))
```

```
                        for g in gènes_desc]
```



```

        genotype_final = ''.join(gènes_orde)
        descendants[genotype_final] = descendants.get(genotype_final, 0) + 1
    total = sum(descendants.values())
    return {k: v/total for k, v in sorted(descendants.items())}

def phenotype_dominant(genotype, allele_dom):
    """Retourne True si le génotype exprime l'allèle dominant."""
    return allele_dom.upper() in genotype

# Monohybridisme : Aa × Aa
print("=== Croisement Aa × Aa ===")
result = punnett('Aa', 'Aa')
print(f"{'Génotype':>8} | {'Fréquence':>10} | {'Phénotype':>12}")
print("-" * 38)
for gen, freq in result.items():
    phen = "Dominant" if 'A' in gen else "Récessif"
    print(f"{gen:>8} | {freq:>10.2%} | {phen:>12}")

# Dihybridisme : AaBb × AaBb
print("\n=== Croisement AaBb × AaBb ===")
result2 = punnett('AaBb', 'AaBb')
print(f"{'Génotype':>10} | {'Fréquence':>10}")
print("-" * 25)
for gen, freq in result2.items():
    print(f"{gen:>10} | {freq:>10.2%}")

# Ratio phénotypique attendu
dom_A = sum(f for g, f in result2.items() if 'A' in g)
dom_B = sum(f for g, f in result2.items() if 'B' in g)
dom_AB = sum(f for g, f in result2.items() if 'A' in g and 'B' in g)
rec_ab = sum(f for g, f in result2.items() if g[0]=='a' and g[2]=='b')
print(f"\nRatio phénotypique A_B : {dom_AB:.0%}")
print(f"Ratio classique dihybride ≈ 9:3:3:1")

```

[PIÈGE]

La loi d'assortiment indépendant (loi 2) ne s'applique que si les gènes sont sur des chromosomes DIFFÉRENTS (ou très éloignés sur le même chromosome). Des gènes proches sur le même chromosome sont "liés" (linkage) et ne s'assortissent pas indépendamment – ils tendent à être transmis ensemble. Les recombinaisons par enjambement (crossing-over) peuvent toutefois séparer des allèles liés.

12. ÉVOLUTION ET GÉNÉTIQUE DES POPULATIONS

[THÉORIE]

La théorie de l'évolution (Darwin, 1859) explique la diversité du vivant par la sélection naturelle. La génétique des populations étudie comment les fréquences alléliques changent dans une population au fil du temps.

[DÉFINITION]

SÉLECTION NATURELLE :

Les individus dont les traits augmentent la survie et la reproduction transmettent davantage leurs allèles aux générations suivantes.

DÉRIVE GÉNÉTIQUE :

Variation aléatoire des fréquences alléliques, plus forte dans les petites populations.

LOI DE HARDY-WEINBERG :

Dans une population idéale (grande, aléatoire, sans mutation, sans migration, sans sélection), les fréquences alléliques et génotypiques restent constantes de génération en génération.

Si p = fréquence de l'allèle A

Et q = fréquence de l'allèle a

Alors $p + q = 1$

Fréquences génotypiques à l'équilibre :

AA : p^2

Aa : $2pq$

aa : q^2

$$\text{Et } p^2 + 2pq + q^2 = 1$$

UTILISATION :

Si la fréquence d'une maladie récessive (aa) est q^2 ,
on peut calculer : $q = \sqrt{q^2}$, puis $p = 1 - q$,
et donc la fréquence des porteurs sains (Aa) = $2pq$.

[DÉMONSTRATION] Mucoviscidose (maladie récessive)

Prévalence en Europe : 1/2500 naissances affectées (aa)

$$q^2 = 1/2500 = 0.0004$$

$$q = \sqrt{0.0004} = 0.02$$

$$p = 1 - 0.02 = 0.98$$

$$\text{Porteurs sains (Aa)} = 2pq = 2 \times 0.98 \times 0.02 = 0.0392 \approx 1/25$$

→ Environ 1 personne sur 25 est porteuse saine de la mucoviscidose. □

[CODE PYTHON]

```
import math
import random

def hardy_weinberg(p):
    """Calcule les fréquences génotypiques à l'équilibre de HW."""
    q = 1 - p
    return {'AA': p**2, 'Aa': 2*p*q, 'aa': q**2, 'p': p, 'q': q}

def maladie_recessive(prevalence):
    """Dédit les fréquences alléliques et porteurs d'une maladie récessive."""
    q2 = prevalence
    q = math.sqrt(q2)
    p = 1 - q
    porteurs = 2 * p * q
    return {'q²(malades)': q2, 'q': q, 'p': p,
            '2pq (porteurs)': porteurs, 'p²(AA)': p**2}

print("=== Hardy-Weinberg – fréquences à l'équilibre ===")
print(f"{'p (A)':>6} | {'q (a)':>6} | {'p²(AA)':>8} | {'2pq(Aa)':>8} | {'q²(aa)':>8}")
print("-" * 46)
for p in [0.1, 0.3, 0.5, 0.7, 0.9, 0.99]:
    hw = hardy_weinberg(p)
    print(f"{'hw['p']':>6.2f} | {'hw['q']':>6.2f} | "
          f"{'hw['AA']':>8.4f} | {'hw['Aa']':>8.4f} | {'hw['aa']':>8.4f}")

print("\n=== Calcul de fréquence de porteurs pour maladies récessives ===")
maladies = {
    'Mucoviscidose': 1/2500,
    'Phénylcétonurie': 1/10000,
    'Albinisme': 1/20000,
    'Drépanocytose': 1/500,
}
print(f"{'Maladie':>20} | {'Prévalence':>12} | {'1 porteur/':>12}")
print("-" * 52)
for nom, prev in maladies.items():
    res = maladie_recessive(prev)
    porteur_ratio = int(round(1 / res['2pq (porteurs)']))
    print(f"{'nom':>20} | {'prev':>12.6f} | 1/{porteur_ratio:<10}")

# Simulation de dérive génétique
print("\n=== Simulation de dérive génétique (N=20, 50 générations) ===")
random.seed(42)
N = 20 # taille de la population
p0 = 0.5
p = p0
print(f"p initial = {p0}")
for gen in range(1, 51):
    # La génération suivante est obtenue par tirage aléatoire
    alleles_A = sum(random.random() < p for _ in range(2*N))
    p = alleles_A / (2*N)
    if gen % 10 == 0:
        barre = "█" * int(p * 30)
        print(f"Génération {gen:>2} : p = {p:.3f} {barre}")
    if p == 0.0 or p == 1.0:
        print(f"Fixation atteinte à la génération {gen} ! p = {p}")
        break
```

[PIÈGE]

Hardy-Weinberg est un MODÈLE NULS – les populations réelles s'en écartent toujours car elles ne sont pas infiniment grandes, subissent des mutations, des migrations et une sélection. L'utilité du modèle est de tester si une population EST en équilibre : un écart à HW indique qu'une force évolutive agit. $p + q = 1$ concerne les FRÉQUENCES ALLÉLIQUES, pas les fréquences génotypiques.

RÉSUMÉ – LES RÈGLES D'OR

CHIMIE GÉNÉRALE

Z = protons = identité de l'élément. $A = Z + N$.

Liaison : $\Delta\chi < 0.5 \rightarrow$ apolaire, $0.5-1.7 \rightarrow$ polaire, $>1.7 \rightarrow$ ionique

STœCHIMÉTRIE

Toujours convertir en moles ($n = m/M$) avant tout calcul.

Réactif limitant $\rightarrow$ celui dont le rapport n/coeff est minimal.

THERMODYNAMIQUE

$\Delta G < 0 \rightarrow$ spontané. $\Delta G = \Delta H - T\Delta S$. Spontanéité $\neq$ rapidité.

$\Delta H < 0 \rightarrow$ exothermique. $\Delta S > 0 \rightarrow$ désordre croissant.

CINÉTIQUE

$v = k[A]^m[B]^n$. Ordre $\neq$ coefficient stœchiométrique.

Arrhenius : k augmente exponentiellement avec T .

ACIDES-BASES

$pH = -\log[H_3O^+]$. $pH + pOH = 14$. Tampon = Henderson-Hasselbalch.

Acide faible : $pH \approx \frac{1}{2}(pK_a - \log C_a)$. Toujours en moles !

BIOLOGIE CELLULAIRE

Procaryote : pas de noyau. Eucaryote : noyau + organites.

Ribosomes 70S (procaryotes) vs 80S (eucaryotes) $\rightarrow$ antibiotiques.

GÉNÉTIQUE MOLÉCULAIRE

ADN $\rightarrow$ ARNm (transcription) $\rightarrow$ protéine (traduction).

Brin matrice 3' $\rightarrow$ 5'. ARNm 5' $\rightarrow$ 3'. U remplace T dans l'ARN.

ENZYMES

K_m faible = haute affinité. V_{max} = vitesse max à saturation.

Compétitif $\rightarrow K_m$ change. Non compétitif $\rightarrow V_{max}$ change.

MÉTABOLISME

Glycolyse (2 ATP) + Krebs + CTE $\approx 30-32$ ATP / glucose.

ATP = monnaie énergétique. $\Delta G(ATP) = -30.5$ kJ/mol.

GÉNÉTIQUE

Mendel : ségrégation et assortiment indépendant.

Hardy-Weinberg : $p + q = 1$, $p^2 + 2pq + q^2 = 1$.

$q = \sqrt{\text{fréquence maladie récessive}}$ $\rightarrow$ calculer les porteurs.

#####

#

CE QUE TU MAÎTRISES MAINTENANT :

#

- # ✓ Structure atomique : numéro atomique Z , configuration électronique
- # ✓ Liaisons chimiques : covalente, ionique, métallique, hydrogène
- # ✓ Stœchiométrie : mole, masse molaire, réactif limitant
- # ✓ Thermodynamique : ΔH , ΔS , ΔG , loi de Hess, Gibbs
- # ✓ Cinétique : loi de vitesse, Arrhenius, équilibre, Le Chatelier
- # ✓ Acides-bases : pH , K_a , pK_a , tampons, Henderson-Hasselbalch
- # ✓ Cellule : procaryote vs eucaryote, organites, membrane
- # ✓ Génétique moléculaire : ADN, ARN, transcription, traduction, code génétique
- # ✓ Enzymes : Michaelis-Menten, K_m , V_{max} , inhibitions
- # ✓ Métabolisme : respiration cellulaire, ATP, photosynthèse
- # ✓ Génétique mendélienne : allèles, carré de Punnett, lois de Mendel
- # ✓ Génétique des pop. : Hardy-Weinberg, dérive génétique, sélection

#

PROCHAINE ÉTAPE NATURELLE :

→ Biochimie (protéines, lipides, glucides, acides nucléiques en détail)
→ Immunologie (système immunitaire, anticorps, lymphocytes)
→ Chimie organique (réactions, mécanismes, groupes fonctionnels)
→ Biologie moléculaire avancée (épigénétique, régulation de l'expression)
→ Électrochimie (piles, électrolyse, potentiel d'électrode)

#####